

Lava Security Hardening

Source-derived overview of Lava's security posture: the Lava-Auth mechanism (client + server), credential handling, TLS posture, the no-hardcoding scanners, and the static-analysis / dependency-scanning tooling. Every statement below is backed by a tracked source file, named inline. Anything that could not be verified from the repository is marked UNCONFIRMED:.

Classification: project-specific.

1. Lava-Auth — per-build encrypted client identity

Lava authenticates the *client app* (not the human user) to the lava-api-go service with a per-build encrypted UUID carried in a single request header. The header field name is **not** hardcoded — it comes from `.env` at build time per §6.R (see §4).

1.1 Client side — AuthInterceptor

Source: `core/network/impl/src/main/kotlin/lava/network/impl/AuthInterceptor.kt`

The OkHttp AuthInterceptor injects the encrypted UUID into each outbound request:

- It reads an encrypted **blob**, **pepper**, **nonce**, and the header **field name** from a LavaAuthBlobProvider (all build-time `.env`-derived, not literals in source).
- The AES-GCM key is derived via HKDF(salt = signingCertHash[:16], ikm = pepper, info = "lava-auth-v1"), with a 32-byte key and a 16-byte salt taken from the APK's signing-cert SHA-256 hash.
- It decrypts the blob to the plaintext UUID ByteArray, Base64-encodes it once into the header value, and sets that header on the request.

Re-signed-APK defence. Because the AES key is salted with the signing-cert hash, a re-signed APK produces a different cert hash → a different derived key → AES-GCM decryption fails with

AEADBadTagException. The interceptor lets that surface as a network error rather than sending a malformed header (per the KDoc on AuthInterceptor).

Fail-closed stub state. When `LavaAuthBlobProvider.getBlob()` returns empty bytes (the Phase-10 stub state described in the KDoc), the interceptor is a no-op pass-through — no header is added.

1.2 Auth UUID memory hygiene

Source: [core/CLAUDE.md](#) — "Auth UUID memory hygiene" section, plus the finally block in `AuthInterceptor.intercept()`.

The decrypted UUID is held **only as a ByteArray**, never as a `String`. Both the derived key bytes and the UUID bytes are zeroized (`fill(0)`) in a finally block before `intercept()` returns. The Base64-encoded header **value** is a `String` (JVM strings are immutable, so it cannot be zeroized), but per `core/CLAUDE.md` it **MUST NOT** be logged, persisted, or assigned to a class field, and a pre-push grep enforces this. The constitution states `AuthInterceptor` is the **only** allowed consumer of the auth blob; reflective access from elsewhere is a violation.

1.3 Server side — AuthMiddleware

Source: [lava-api-go/internal/auth/middleware.go](#)

The Gin middleware (`NewMiddleware`) enforces Lava-Auth on every request that flows through it. The field name and HMAC secret come from config (`cfg.AuthFieldName`, `cfg.AuthHMACSecret`), not literals.

Per-request behaviour (from the source):

Condition	Response	Backoff effect
Missing header	401 Unauthorized	none
Malformed / non-Base64 / empty blob	401 Unauthorized	<code>ladder.RecordFailure(ip)</code>
Hash in the active allowlist	<code>c.Next()</code> + sets <code>client_name</code>	<code>ladder.Reset(ip)</code>
Hash in the retired allowlist	426 Upgrade Required + min-version JSON	none (honest-but-outdated client)
Unknown hash	401 Unauthorized	<code>ladder.RecordFailure(ip)</code>

Key hardening details verified in source:

- **HMAC identity.** The server stores `hex(HMAC-SHA256(blob, secret))`. The raw UUID is never persisted server-side — only its keyed hash (`hashUUIDBlob`).
- **Constant-time lookup.** `constantTimeMapLookup` iterates **all** allowlist entries on every call using `subtle.ConstantTimeCompare`, deliberately defeating the timing side-channel a short-circuiting map lookup would leak (which buckets the attacker's hash collides with).
- **Blob zeroization (§6.H).** The decoded blob bytes are zeroized (`blob[i] = 0`) immediately after hashing.
- **Backoff coupling.** The middleware advances a shared `*ladder.Ladder` (from `digital.vasic.ratelimiter`) on failure; the same Ladder instance is consumed by a `BackoffMiddleware` (see `lava-api-go/internal/auth/backoff.go`) so repeated failures from an IP escalate into 429-style responses on later requests.

The retired-client path returns a JSON body with `min_supported_version_name` and `min_supported_version_code`, letting an outdated client know it must upgrade rather than being treated as an attacker.

2. Credential handling

Lava's constitution (§6.H, Credential Security Inviolability) forbids any real credential, signing key, or API secret from ever appearing in a tracked file. The following are verified gitignored in `.gitignore`:

Path	Contains	.gitignore line
<code>.env, .env.*, .env.local</code>	All runtime secrets (auth field name, pepper, HMAC secret, tracker credentials, tokens)	12, 52, 53
<code>keystores/</code>	APK signing keys	13
<code>app/google-services.json, **/google-services.json</code>	Firebase project web API key	59, 60
<code>lava-api-go/firebase-web-config.json</code>	Firebase Web SDK config	65
<code>lava-api-go/firebase-admin-key.json, **/firebase-admin-*.json</code>	Firebase Admin SDK service-account key (long-lived, highly sensitive)	70–72

Path	Contains	.gitignore line
firebase-debug.log, **/firebase-debug.log	May leak request bodies / OAuth tokens	76, 77
app/build/generated/lava-auth/	Build-time generated lava.auth.LavaAuthGenerated source	84
.lava-ci-evidence/**/logcat-*.txt, .lava-ci-evidence/**/post-mortem/	logcat captures may contain credentials	89, 90

.env.example is the committed placeholder file carrying dummy values for every variable a developer must set (per §6.R / §6.H).

Per §6.H the project also runs a **pre-push credential scan** in scripts/check-constitution.sh that rejects pushes introducing .env* / keystore files or credential-pattern lines into tracked files.

3. TLS / transport posture

Sources: [app/src/main/AndroidManifest.xml](#), [app/src/main/res/xml/network_security_config.xml](#).

- **android:usesCleartextTraffic is true** (verified in the manifest, line 48). This is deliberate: the **legacy LAN Ktor proxy** is reachable over cleartext `http://<lan-ip>:8080`. Public-internet endpoints (rutracker.org and mirrors) and the lava-api-go HTTPS path do not rely on cleartext.
- **android:networkSecurityConfig="@xml/network_security_config"** is set (manifest line 49). The NSC file:
 - `base-config cleartextTrafficPermitted="true"` with the **system** trust store as the trust anchor — the strict path for public-internet TLS.
 - A `domain-config cleartextTrafficPermitted="true"` allow-list scoped to LAN hosts (localhost, 127.0.0.1, *.local for mDNS, and common home-LAN gateway IPs) used only by the legacy-proxy code path.
- **lava-api-go HTTPS on port 8443** does **not** go through NSC trust-anchor resolution. Per the NSC file comments, the LAN HTTPS path is handled at the OkHttp layer via a LAN-permissive `@Named("lan") OkHttpClient` (see the KDoc in `core/network/impl/.../NetworkModule.kt`).
 - The previous "user installs the self-signed CA via device Settings" path was **removed** in SP-3.1 (2026-04-29) because it required a manual device-settings detour

forbidden by Sixth-Law clause 4. (NSC file header comment.)

- The on-device Lava API app serves **HTTPS** with a certificate it generates on the device on first run; see [docs/guides/ON_DEVICE_API_USER_GUIDE.md](#).
- The :app ↔ on-device-API key handoff uses a **signature-level Android permission** (`digital.vasic.lava.permission.READ_API_KEY`, manifest line 9): because both apps share the same keystore, only the genuine Lava client is granted access to the API app's key ContentProvider at install time.

4. No-Hardcoding Mandate — scanners

§6.R forbids any connection address, port, header field name, credential, key, salt, secret, or domain literal in tracked source. Three standalone scanners enforce subsets of this and are invoked by `scripts/check-constitution.sh` / the pre-push hook:

Scanner	Detects	Source
<code>scripts/scan-no-hardcoded-uuid.sh</code>	36-char UUID literals	link
<code>scripts/scan-no-hardcoded-ipv4.sh</code>	IPv4 literals (loopback / RFC 5737 docs IPs filtered out)	link
<code>scripts/scan-no-hardcoded-hostport.sh</code>	<code>scheme://host:port</code> URL literals (<code>localhost/127.x/0.0.0.0</code> filtered out)	link

Each scanner exits 1 (and prints offending paths to `stderr`) on a violation. Shared exemptions, kept in lockstep with the §6.R clause body, include: `.env.example`, `.lava-ci-evidence/`, `submodules/` (pinned vendored code), test sources (`*_test.go`, `*Test.kt`, `*Tests.kt`, `*Test.java`, `src/test/`, `src/androidTest/`, `fixtures/`), and external config/docs file types (`*.md`, `*.json`, `*.xml`, `*.yaml`, `*.yml`) — because config files like `network_security_config.xml` are the legitimate home for connection literals; §6.R targets the *code that reads them*.

Per the §6.R clause, the schedule and algorithm-parameter literal classes (e.g. PBKDF2 iteration counts, AES key sizes) remain a code-review gate rather than a regex scanner, because they are indistinguishable from ordinary integers by pattern alone.

5. Static analysis & dependency scanning

The completeness program (Phase 2) wired containerized, rootless, local-only scanning. See the per-tool triage docs in this directory:

- **Detekt** (Kotlin static analysis) — [2026-06-04-detekt-triage.md](#). Wired through the `StaticAnalysisConventionPlugin` `buildSrc` convention so every module is analysed with no per-module config (`io.gitlab.arturbosch.detekt 1.23.8`). Per-module baselines capture today's 674 deduplicated finding IDs (`build.maxIssues: 0`), so the gate is green going forward while any **new** finding fails the build.
- **Go quality gate** (`go vet + golangci-lint + coverage`) — [2026-06-04-golangci-triage.md](#). `go vet` is clean (0 findings); `golangci-lint` runs containerized via rootless podman (`scripts/golangci-lint.sh`) against a curated `.golangci.yml` and passes with 0 issues after curated excludes.
- **SonarQube + Snyk** (containerized) — [2026-06-04-sonarqube-snyk-setup.md](#). Rootless SonarQube CE + dedicated Postgres (bound to `127.0.0.1` only, no `--privileged`, no host `sysctl`) via `docker-compose.sonar.yml + scripts/sonar-scan.sh`; Snyk dependency + SAST via `scripts/snyk-scan.sh`. Tokens (`SONAR_TOKEN`, `SNYK_TOKEN`) are read from `.env` only. Per §6.J/§6.Z the scripts exit non-zero (never a fabricated "passed") when a token is absent or the server is not UP. See the triage doc for the precise ran-here vs. operator-blocked breakdown.

All scanning is **local-only** by constitutional constraint — no hosted CI (GitHub Actions, GitLab pipelines, etc.) is used; the same scripts a developer runs locally are the gate.

6. Constitutional cross-references

- §6.H — Credential Security Inviolability (root `CLAUDE.md`)
- §6.R — No-Hardcoding Mandate (root `CLAUDE.md`)
- §6.U — No sudo/su Mandate (root `CLAUDE.md`)
- "Auth UUID memory hygiene" — `core/CLAUDE.md`
- "Local-Only CI/CD" — root `CLAUDE.md`